

Tabak-Trigila Notes

James Evans

08/01/2025

Monge Formulation

The optimal transport problem posed by Monge is to find a plan to transport material from one location to another that minimizes the total transportation cost. By normalizing the total amount of material to transport to one, the initial and final distributions can be modeled as two probability densities, thus stating the transport problem in the following terms:

Given two probability densities $\rho(x)$ and $\mu(y)$ with $x, y \in \mathbb{R}^n$, find the map $y = y(x) : \mathbb{R}^n \rightarrow \mathbb{R}^n$ transporting the density $\rho(x)$ to $\mu(y)$ that minimizes the cost functional

$$M(y) = \int_{\mathbb{R}^n} c(x, y(x)) \rho(x) dx. \quad (1)$$

In order to transport $\rho(x)$ to $\mu(y)$ the map is required to satisfy the relation

$$\int_{y^{-1}(\Omega)} \rho(x) dx = \int_{\Omega} \mu(y) dy \quad (2)$$

for all measurable sets Ω . If $y(x)$ is smooth and one to one, this is equivalent to the point-wise relation

$$\rho(x) = J^y(x) \mu(y(x)), \quad (3)$$

where $J^y(x)$ is the Jacobian of the map: $J^y(x) = \det(\nabla y(x))$. A function $y(x)$ satisfying (3) and minimizing $M(y)$ is called an *optimal map*.

Kantorovich Formulation

We begin with the Monge cost functional:

$$M(y) = \int c(x, y(x)) \rho(x) dx$$

where $\rho(x)$ is the source density and $y(x)$ is a deterministic transport map. From the Kantorovich formulation, the marginal constraint is:

$$\rho(x) = \int \pi(x, y) dy$$

where $\pi(x, y)$ is a joint transport plan coupling $\rho(x)$ and the target density $\mu(y)$. First, substitute the expression for $\rho(x)$ into the Monge functional:

$$M(y) = \int c(x, y(x)) \left(\int \pi(x, y) dy \right) dx$$

Since $c(x, y(x))$ does not depend on y , it can be pulled inside the inner integral:

$$\int \left[c(x, y(x)) \cdot \left(\int \pi(x, y) dy \right) \right] dx = \int \int c(x, y(x)) \pi(x, y) dy dx$$

This uses the basic fact that for any scalar function $f(x)$:

$$f(x) \cdot \int g(x, y) dy = \int f(x) g(x, y) dy$$

Fubini's Theorem states that if the integrand is measurable (the function behaves well enough that areas under its curve can be calculated) and the integral of its absolute value is finite, then we may exchange the order of integration:

$$\int \left(\int f(x, y) dy \right) dx = \int \left(\int f(x, y) dx \right) dy$$

Applying this to the expression above yields:

$$M(y) = \int \int c(x, y(x)) \pi(x, y) dx dy$$

In the Kantorovich setting, the transport is no longer restricted to deterministic maps $y = y(x)$, so we relax $c(x, y(x))$ to $c(x, y)$, yielding the Kantorovich functional:

$$K(\pi) = \int \int c(x, y) \pi(x, y) dx dy$$

This is the relaxed version of the Monge cost, now optimized over all joint couplings $\pi(x, y)$ that satisfy the marginal constraints:

$$\int \pi(x, y) dy = \rho(x), \quad \int \pi(x, y) dx = \mu(y)$$

Monge's formulation of the transport problem requires a one-to-one map between the points at the origin and destination. Kantorovich proposed a relaxation where the mass reaching each point y may come from various points x and, conversely, the mass from each point x may be split into various destinations y . In this formulation, the way in which ρ must be rearranged to obtain μ is described by a joint probability density $\pi(x, y) \geq 0$ satisfying

$$\rho(x) = \int \pi(x, y) dy, \quad \mu(y) = \int \pi(x, y) dx \tag{4}$$

and the problem reduces to minimizing the functional

$$K(\pi) = \int c(x, y) \pi(x, y) dx dy. \tag{5}$$

A minimizer of (5) subject to (4) is called an *optimal plan*. It can be shown that under relatively mild hypotheses, such as $\rho(x)$ and $\mu(y)$ having finite second moments and $c(x, y)$ being a strictly convex function of $y - x$ ([20]), the relaxed problem and the original Monge problem have the same unique solution, in the sense that

$$\min M(y) = \min K(\pi) \quad (6)$$

and the minimizers satisfy

$$\pi(S) = \rho(\{x : (x, y(x)) \in S\}). \quad (7)$$

Notice that unlike Monge's formulation, in which the cost function and the constraints are nonlinear in the unknown $y(x)$, in Kantorovich's both are linear in $\pi(x, y)$, yielding a continuous version of the *assignment problem*, a category of linear programming problems with wide applications in economics. Then it has a dual formulation, which adopts the form:

$$D(u, v) = \max_{u, v} \int u(x) \rho(x) dx + \int v(y) \mu(y) dy \quad (8)$$

over all integrable functions u and v satisfying

$$u(x) + v(y) \leq c(x, y). \quad (9)$$

Dual Formulation of Kantorovich Optimal Transport

The Kantorovich **primal problem** is:

$$\min_{\pi} \int c(x, y) \pi(x, y) dx dy$$

subject to:

$$\int \pi(x, y) dy = \rho(x), \quad \int \pi(x, y) dx = \mu(y)$$

This finds the transport plan $\pi(x, y)$ that moves mass from $\rho(x)$ to $\mu(y)$ at minimum total cost. The **dual problem** is:

$$\max_{u, v} \left[\int u(x) \rho(x) dx + \int v(y) \mu(y) dy \right]$$

subject to the inequality constraint:

$$u(x) + v(y) \leq c(x, y) \quad \text{for all } x, y$$

What this means:

- You are optimizing over all real-valued functions $u(x)$ and $v(y)$.

- The constraint $u(x) + v(y) \leq c(x, y)$ must hold for every pair (x, y) . This means that the sum $u(x) + v(y)$ is allowed to *underestimate* $c(x, y)$ but never exceed it.
- The objective is to make the quantity

$$\int u(x) \rho(x) dx + \int v(y) \mu(y) dy$$

as large as possible while still satisfying the constraint. This dual problem gives a lower bound on the optimal transport cost. When certain regularity conditions hold (like c being continuous and ρ, μ having finite second moments), the bound is tight:

$$\min_{\pi} \int c(x, y) \pi(x, y) dx dy = \max_{u, v} \left[\int u(x) \rho(x) dx + \int v(y) \mu(y) dy \right]$$

In other words, solving either the primal or the dual problem gives the same optimal transport cost.

Linearity in the Kantorovich Formulation

Notice that unlike Monge's formulation, in which both the cost and constraints are nonlinear in the unknown function $y(x)$, the Kantorovich formulation is linear in the decision variable $\pi(x, y)$.

Monge's formulation:

$$\min_{y(x)} \int c(x, y(x)) \rho(x) dx$$

- The unknown is the transport map $y(x)$.
- The cost functional depends on $y(x)$ through $c(x, y(x))$, which is generally nonlinear (in $y(x)$).
- Additionally, the constraint that y pushes forward ρ to μ (i.e., $y_{\#}\rho = \mu$) is also nonlinear.
- Therefore, Monge's formulation is a nonlinear optimization problem.

Kantorovich's formulation:

$$\min_{\pi \geq 0} \int c(x, y) \pi(x, y) dx dy$$

subject to the linear marginal constraints:

$$\int \pi(x, y) dy = \rho(x), \quad \int \pi(x, y) dx = \mu(y)$$

- The unknown is the transport plan $\pi(x, y)$.
- The cost function is linear in $\pi(x, y)$.
- The constraints are linear in $\pi(x, y)$ as well.
- As a result, the Kantorovich formulation is a linear program over functions.

Note: In the dual formulation, $u(x)$ and $v(y)$ represent source and destination costs, respectively. The constraint $u(x) + v(y) \leq c(x, y)$ ensures these costs do not overestimate the true transport cost, preventing any double counting.

🔍 Let's look at the objective:

$$\min_{\pi \geq 0} \int c(x, y) \pi(x, y) dx dy$$

Here:

- $c(x, y)$ is fixed — you're not optimizing it.
- $\pi(x, y)$ is the variable.
- You are integrating a function that is **just a constant times** $\pi(x, y)$.

So this is **linear in π** — there are **no powers**, **no products like π^2** , **no nonlinear transformations** (e.g. $\log(\pi)$, etc.)

That makes the **cost functional** linear.

Linearity means: if you scale or add two transport plans π_1, π_2 , the objective and constraints scale or add in the same way.

$$\bullet \text{ Marginals: } \int \pi dy = a \cdot \int \pi_1 dy + b \cdot \int \pi_2 dy$$

Connection to the Assignment Problem:

The classical assignment problem asks for the cheapest way to assign n workers to n jobs, given a cost matrix $C = (c_{ij})$. The discrete version of the problem is:

$$\min_P \sum_{i,j} c_{ij} P_{ij} \quad \text{subject to } P \text{ is a permutation matrix}$$

The Kantorovich formulation is a *continuous version* of this problem:

- Replace discrete indices i, j with continuous variables x, y .
- Replace the matrix P with a joint density function $\pi(x, y)$.
- Still minimize total transport cost, under marginal constraints.

Thus, Kantorovich's problem can be interpreted as a **continuous assignment problem** — a linear programming problem with wide applications in economics, optimization, and operations research.

A Data-Based Formulation

Optimal transport has a broad range of applications in mathematics and beyond, in fields as diverse as image processing and economics. As mentioned above, many more applications arise if one extends the problem to situations in which the two densities $\rho(x)$ and $\mu(y)$ are not known pointwise but through samples: m points $\{x_i\}$ drawn from $\rho(x)$ and n points $\{y_j\}$ drawn from $\mu(y)$.

In order to develop a formulation of the optimal transport problem suited to such data-driven generalization, we notice that in Kantorovich dual formulation (8) the probability densities $\rho(x)$ and $\mu(y)$ appear only in the objective function D , and in a form that can be interpreted in terms of expected values:

$$D(u, v) = E[u(x)]_{\rho(x)} + E[v(y)]_{\mu(y)}. \quad (14)$$

Thus, if $\rho(x)$ and $\mu(y)$ are known only through samples, it is natural to pose the problem in terms of sample averages:

Maximize

$$D(u, v) = \frac{1}{m} \sum_{i=1}^m u(x_i) + \frac{1}{n} \sum_{j=1}^n v(y_j)$$

over functions u and v satisfying

$$u(x) + v(y) \leq c(x, y).$$

In order to complete the statement of this problem, one needs to specify the space of functions to which u and v belong. As we shall see below, various possibilities arise. Moreover, since this data-based formulation is presented in terms of the dual variables u and v , the meaning of its solution and its relation to the original transport problem must be described. These two kinds of considerations are closely interlinked; we address them in section 2.

In the dual formulation of optimal transport, expectations are often written using subscript notation to indicate the distribution with respect to which the expectation is taken. For example:

$$\mathbb{E}_\rho[u(x)] = \int u(x) \rho(x) dx, \quad \text{and} \quad \mathbb{E}_\mu[v(y)] = \int v(y) \mu(y) dy$$

Here, ρ and μ denote the source and target probability distributions, respectively. The subscript notation $\mathbb{E}_\rho$ indicates that the expectation is taken over the random variable $x \sim \rho$, while $\mathbb{E}_\mu$ indicates the expectation over $y \sim \mu$. Thus, the Kantorovich dual objective function:

$$D(u, v) = \int u(x) \rho(x) dx + \int v(y) \mu(y) dy$$

can be equivalently written in expectation notation as:

$$D(u, v) = \mathbb{E}_\rho[u(x)] + \mathbb{E}_\mu[v(y)]$$

This notation becomes especially useful when transitioning to a data-driven formulation, where these expectations are approximated by sample averages.

Discrete and Continuous Data-Based Formulations of OT

Therefore we return to the fully constrained formulation in (16). Since this is a Monte Carlo relaxation of the dual to the problem of interest, we first find its own dual. In terms of the Lagrangian

$$L(\pi, u, v) = \frac{1}{m} \sum_{i=1}^m u(x_i) + \frac{1}{n} \sum_{j=1}^n v(y_j) - \int \pi(x, y) [u(x) + v(y) - c(x, y)] dx dy,$$

the problem in (15, 16) can be formulated as

$$d : \max_{u(x), v(y)} \min_{\pi(x, y) \geq 0} L(\pi, u, v), \quad (22)$$

and its dual as

$$p : \min_{\pi(x, y) \geq 0} \max_{u(x), v(y)} L(\pi, u, v). \quad (23)$$

(We adopt the reverse notation d and p for primal and dual since these relate to the dual and primal of the original transportation problem). To study p , it is convenient to rewrite the Lagrangian in the more revealing form:

We begin with the sample-based dual formulation of optimal transport, where the goal is to maximize:

$$D(u, v) = \frac{1}{m} \sum_{i=1}^m u(x_i) + \frac{1}{n} \sum_{j=1}^n v(y_j) \quad \text{subject to} \quad u(x_i) + v(y_j) \leq c(x_i, y_j)$$

To encode the inequality constraint, we introduce a Lagrange multiplier $\pi(x, y) \geq 0$ and define the Lagrangian:

$$L(\pi, u, v) = \frac{1}{m} \sum_{i=1}^m u(x_i) + \frac{1}{n} \sum_{j=1}^n v(y_j) - \int \pi(x, y) [u(x) + v(y) - c(x, y)] dx dy$$

This formulation penalizes constraint violations in the objective function. Using this Lagrangian, we can express the original optimization problem as a saddle-point problem.

Equation (22): Dual Formulation

$$d := \max_{u(x), v(y)} \min_{\pi(x, y) \geq 0} L(\pi, u, v)$$

Here:

- The outer maximization is over the dual potentials $u(x)$ and $v(y)$.
- The inner minimization is over transport plans $\pi(x, y) \geq 0$.
- This corresponds to the dual (Kantorovich) formulation of optimal transport.

Intuitively, this saddle-point problem works as follows:

- The term $\pi(x, y)[u(x) + v(y) - c(x, y)]$ becomes positive when the constraint is violated, and zero or negative when it is satisfied.
- Because this term is subtracted in the Lagrangian, minimizing over $\pi \geq 0$ makes the total penalty as large (i.e., as harmful) as possible — this is the worst-case penalty against your chosen u, v .
- Once this worst-case penalty is accounted for, you then maximize over u, v to find the best possible values that still make the Lagrangian large, even under those penalties.

Equation (23): Primal Formulation

$$p := \min_{\pi(x, y) \geq 0} \max_{u(x), v(y)} L(\pi, u, v)$$

Here:

- The outer minimization is over admissible transport plans $\pi(x, y) \geq 0$.
- The inner maximization is over dual potentials $u(x), v(y)$.
- This corresponds to the primal (Monge) problem: find the optimal plan π that minimizes cost.

This exchange of the order of min and max is justified by the **Minimax Theorem**, which states that if the function $L(\pi, u, v)$ is convex in π and concave in (u, v) , and the feasible sets are convex and compact, then:

$$\max_{u, v} \min_{\pi} L(\pi, u, v) = \min_{\pi} \max_{u, v} L(\pi, u, v)$$

This allows us to pass from the dual to the primal and vice versa.

$$L(\pi, u, v) = \int c(x, y) \pi(x, y) dx dy \quad (24)$$

$$- \int \left[\int \pi(x, y) dy - \frac{1}{m} \sum_{i=1}^m \delta(x - x_i) \right] u(x) dx \quad (25)$$

$$- \int \left[\int \pi(x, y) dx - \frac{1}{n} \sum_{j=1}^n \delta(y - y_j) \right] v(y) dy. \quad (26)$$

We begin with the original Lagrangian:

$$L(\pi, u, v) = \frac{1}{m} \sum_{i=1}^m u(x_i) + \frac{1}{n} \sum_{j=1}^n v(y_j) - \int \pi(x, y) [u(x) + v(y) - c(x, y)] dx dy$$

We split the integral into three terms:

$$\int \pi(x, y) u(x) dx dy + \int \pi(x, y) v(y) dx dy - \int \pi(x, y) c(x, y) dx dy$$

The Lagrangian becomes:

$$L(\pi, u, v) = \left[\frac{1}{m} \sum_{i=1}^m u(x_i) - \int \pi(x, y) u(x) dx dy \right] + \left[\frac{1}{n} \sum_{j=1}^n v(y_j) - \int \pi(x, y) v(y) dx dy \right] + \int \pi(x, y) c(x, y) dx dy$$

Delta Function Trick: We now rewrite the discrete sums $\sum u(x_i)$ and $\sum v(y_j)$ as integrals using the Dirac delta function.

To rewrite the discrete sums $\sum u(x_i)$ and $\sum v(y_j)$ as integrals, we use the **Dirac delta function** $\delta(x - x_i)$, which satisfies:

$$\int f(x) \delta(x - x_i) dx = f(x_i)$$

This lets us turn a sum into an integral against a delta spike. But to go from the full sum to an integral expression, we also rely on the **linearity of the integral**. Specifically:

$$\int \left(\sum_{i=1}^m a_i(x) \right) dx = \sum_{i=1}^m \int a_i(x) dx$$

We apply this to rewrite the sum:

$$\frac{1}{m} \sum_{i=1}^m u(x_i) \quad \text{as} \quad \int \left(\frac{1}{m} \sum_{i=1}^m \delta(x - x_i) \right) u(x) dx$$

Here's why it works:

$$\begin{aligned} \int \left(\frac{1}{m} \sum_{i=1}^m \delta(x - x_i) \right) u(x) dx &= \frac{1}{m} \sum_{i=1}^m \int \delta(x - x_i) u(x) dx \\ &= \frac{1}{m} \sum_{i=1}^m u(x_i) \end{aligned}$$

This shows that the delta-based expression is mathematically identical to the original sum. It simply re-expresses the discrete empirical distribution $\mu_{\text{empirical}} = \frac{1}{m} \sum_i \delta(x - x_i)$ in integral form.

The same reasoning applies to the sum $\frac{1}{n} \sum_j v(y_j)$, replacing it with:

$$\int \left(\frac{1}{n} \sum_{j=1}^n \delta(y - y_j) \right) v(y) dy$$

This is an integral against the empirical distribution supported on the samples $x_1, \dots, x_m$. Similarly, we rewrite:

$$\frac{1}{n} \sum_{j=1}^n v(y_j) = \int \left(\frac{1}{n} \sum_{j=1}^n \delta(y - y_j) \right) v(y) dy$$

Substituting back, the Lagrangian becomes:

$$L(\pi, u, v) = \int c(x, y) \pi(x, y) dx dy \tag{24}$$

$$- \int \left[\int \pi(x, y) dy - \frac{1}{m} \sum_{i=1}^m \delta(x - x_i) \right] u(x) dx \tag{25}$$

$$- \int \left[\int \pi(x, y) dx - \frac{1}{n} \sum_{j=1}^n \delta(y - y_j) \right] v(y) dy \tag{26}$$

- Equation (24) is the total transport cost under plan π .
- Equation (25) compares the mass sent out from each x (under π) to the empirical mass available at x_i .

- Equation (26) compares the mass arriving at each y to the empirical mass assigned to each y_j .

Remember: Empirical mass is how we approximate the integral (or expectation) using sample data.

If π is a valid transport plan that respects the sample marginals, the mismatch terms in (25) and (26) vanish, and the Lagrangian reduces to just the cost term. Otherwise, the mismatch terms show how far π deviates from the empirical distribution.

Then, if the functions $u(x)$ and $v(y)$ are only constrained to be integrable, the problem p becomes

$$p : \min_{\pi(x,y) \geq 0} \int c(x,y) \pi(x,y) dx dy \quad (27)$$

subject to

$$\int \pi(x,y) dy = \frac{1}{m} \sum_{i=1}^m \delta(x - x_i), \quad \int \pi(x,y) dx = \frac{1}{n} \sum_{j=1}^n \delta(y - y_j), \quad (28)$$

Derivation of the Empirical Distribution Using the Dirac Delta

For any test function f , the true expectation is:

$$\mathbb{E}_\rho[f] = \int f(x) \rho(x) dx.$$

Using the samples, we approximate the expectation as:

$$\frac{1}{m} \sum_{i=1}^m f(x_i).$$

Using the identity

$$\int f(x) \delta(x - x_i) dx = f(x_i),$$

we can write the sum as:

$$\frac{1}{m} \sum_{i=1}^m f(x_i) = \frac{1}{m} \sum_{i=1}^m \int f(x) \delta(x - x_i) dx = \int f(x) \left(\frac{1}{m} \sum_{i=1}^m \delta(x - x_i) \right) dx.$$

We define:

$$\hat{\rho}(x) := \frac{1}{m} \sum_{i=1}^m \delta(x - x_i).$$

Then the Monte Carlo estimate becomes:

$$\frac{1}{m} \sum_{i=1}^m f(x_i) = \int f(x) \hat{\rho}(x) dx.$$

Since

$$\int f(x) \hat{\rho}(x) dx \approx \int f(x) \rho(x) dx \quad \text{for all } f,$$

we conclude that

$$\rho(x) \approx \frac{1}{m} \sum_{i=1}^m \delta(x - x_i) \quad (\text{in the sense of distributions}).$$

Remember: $\rho(x) = \int \pi(x, y) dy$.

Note: This formula for $\rho(x)$ can also be derived from the fact that we need the following expression to be zero:

$$\left[\int \pi(x, y) dy - \frac{1}{m} \sum_{i=1}^m \delta(x - x_i) \right].$$

This expression is also expressed at the constraint in equation (28).

Constrained to be Integrable

We are given the saddle-point formulation:

$$p := \min_{\pi(x, y) \geq 0} \max_{u(x), v(y)} L(\pi, u, v),$$

where $L(\pi, u, v)$ is the Lagrangian that depends on all three variables.

We are trying to choose functions $u(x)$ and $v(y)$ that make the Lagrangian as large as possible. Importantly:

- $u(x)$ and $v(y)$ appear linearly in $L(\pi, u, v)$
- So unless something constrains them, the maximization can go to $+\infty$

The only thing that prevents the Lagrangian from blowing up is when the following marginal constraints are satisfied:

$$\int \pi(x, y) dy = \frac{1}{m} \sum_{i=1}^m \delta(x - x_i), \quad \int \pi(x, y) dx = \frac{1}{n} \sum_{j=1}^n \delta(y - y_j)$$

If these are not satisfied, the maximization over u and v will diverge. So the maximization effectively **forces** these constraints to hold.

Once the constraints are enforced, the maximization step is done. The resulting problem is:

$$\min_{\pi(x, y) \geq 0} \int c(x, y) \pi(x, y) dx dy$$

subject to:

$$\int \pi(x, y) dy = \frac{1}{m} \sum_{i=1}^m \delta(x - x_i), \quad \int \pi(x, y) dx = \frac{1}{n} \sum_{j=1}^n \delta(y - y_j)$$

This is exactly the primal optimal transport problem with empirical measures as marginals (Equations 27 and 28 in the notes).

Note: When the notes say $u(x)$ and $v(y)$ are “only constrained to be integrable,” they mean:

$$\int |u(x)| dx < \infty, \quad \int |v(y)| dy < \infty$$

No additional constraints are placed — such as boundedness, convexity, or the condition $u(x) + v(y) \leq c(x, y)$. This freedom allows the dual potentials to enforce the marginal constraints on $\pi(x, y)$ through the Lagrangian maximization step.

Question: Can the following Lagrangian terms:

$$\int \left[\int \pi(x, y) dy - \frac{1}{m} \sum_{i=1}^m \delta(x - x_i) \right] u(x) dx, \quad \int \left[\int \pi(x, y) dx - \frac{1}{n} \sum_{j=1}^n \delta(y - y_j) \right] v(y) dy$$

become arbitrarily large, even if $u(x)$ and $v(y)$ are only constrained to be integrable?

Answer: Yes. Here is the key idea:

Assume that the residual error term is supported only on a small measurable subset $A \subset \mathbb{R}^d$, and that $r(x) > 0$ for all $x \in A$. Let A have measure $|A| = \varepsilon > 0$ (A occupies a small but nonzero region of space). Define the test function $u(x)$ as:

$$u(x) = \begin{cases} \frac{1}{\varepsilon} & \text{if } x \in A, \\ 0 & \text{otherwise.} \end{cases}$$

Then:

$$\int |u(x)| dx = \int_A \frac{1}{\varepsilon} dx = \frac{1}{\varepsilon} \cdot \varepsilon = 1,$$

so the integrability condition is satisfied. However, the term:

$$\int r(x) u(x) dx = \int_A r(x) \cdot \frac{1}{\varepsilon} dx = \frac{1}{\varepsilon} \int_A r(x) dx$$

can be made arbitrarily large as $\varepsilon \rightarrow 0$, because the denominator vanishes while the numerator remains strictly positive. Thus, even though $u(x) \in L^1$, the product $r(x)u(x)$ can yield a divergent integral.

namely the Kantorovich formulation of the (primal) optimal transport problem with discrete densities $\rho(x)$ and $\mu(y)$ as in (28). This is not a surprising result, since the problem d is precisely the dual to this problem. Yet this result poses a dilemma: our motivation was not to consider a discrete transport problem that assigns points from y_j to x_i , but one where the x_i and y_j are samples from hidden, presumably smooth distributions $\rho(x)$ and $\mu(y)$. How can one formulate a problem with this flavor?

A natural answer involves restricting the space of functions F from where u and v can be selected. If the space F is invariant under multiplication by scalars:

$$u \in F, \lambda \in \mathbb{R} \rightarrow \lambda u \in F,$$

then the problem p becomes

$$p : \min_{\pi(x,y) \geq 0} \int c(x,y) \pi(x,y) dx dy \quad (29)$$

subject to

$$\int \left[\int \pi(x,y) dy - \frac{1}{m} \sum_{i=1}^m \delta(x - x_i) \right] u(x) dx = 0, \quad (30)$$

$$\int \left[\int \pi(x,y) dx - \frac{1}{n} \sum_{j=1}^n \delta(y - y_j) \right] v(y) dy = 0 \quad (31)$$

for all $u(x), v(y) \in F$. This weak formulation of the strong constraints in (28) is quite natural: we constrain the marginals $\rho(x)$ and $\mu(y)$ of $\pi(x,y)$ so that the corresponding expected values of all functions in F agree with their averages over the samples x_i and y_j respectively.

In equation (28), the marginal constraints were **enforced directly**:

$$\int \pi(x,y) dy = \frac{1}{m} \sum_{i=1}^m \delta(x - x_i) \quad (\text{must hold pointwise})$$

This means:

- For **every value of** x , the total mass assigned to it from y must equal the empirical measure (a sum of Dirac deltas).
- This is a **hard constraint**: it's **required to hold exactly**, for all x .

Same for the y -marginals.

In equation (30), we now see:

$$\int \left[\int \pi(x,y) dy - \frac{1}{m} \sum_{i=1}^m \delta(x - x_i) \right] u(x) dx = 0$$

Let's call the term in brackets the **error in the marginal**:

$$\text{Error}(x) := \int \pi(x,y) dy - \frac{1}{m} \sum_{i=1}^m \delta(x - x_i)$$

We are now asking:

$$\int \text{Error}(x) u(x) dx = 0 \quad \text{for all } u \in F$$

This means: the error does **not need to be zero pointwise**. It just needs to **disappear when integrated against every function** $u \in F$.

This is a strictly weaker requirement:

- Before: the marginal error had to be zero **for every point** x .

- Now: the error only needs to be **orthogonal** to all test functions $u \in F$.

The set F is a chosen function space. We don't force exact agreement — only that the mismatch is invisible when tested through F .

This condition is just like a **dot product**:

$$\langle \text{Error}, u \rangle = \int \text{Error}(x) u(x) dx = 0$$

This tells us the error is **orthogonal** to all functions in F . It's not required to be zero; just invisible when projected onto F . That's why they ask for:

$$u \in F, \lambda \in \mathbb{R} \Rightarrow \lambda u \in F$$

so the space is closed under scaling — a key property for linearity and duality.

- **Function spaces are vector spaces.**
- But they are often **infinite-dimensional**, and elements are functions instead of tuples.
- The concepts of **orthogonality**, **dot product**, and **projection** still apply — just with integrals.

$$\text{Dot product: } \sum_i v_i w_i \quad \longrightarrow \quad \int f(x) g(x) dx$$

The choices above appear sensible yet none is completely natural, since the specification of F or the qualification of smoothness that one seeks should depend on the samples $\{x_i\}$, $\{y_j\}$ themselves: one can allow a richer set of functions in areas where there are enough sample points to fully determine them

than in those where the samples are sparse. In other words, one must restrict F , in order not to over-resolve the densities $\rho(x)$ and $\mu(y)$, to smooth functions with scales long compared to the distance between neighboring sample points, a concept analogous to the bandwidth of kernel density estimation [17]. Again, this can be done either through a careful choice of a basis for F or through an appropriate penalization term that considers the underlying densities. In either case, the choice of F must be adaptive (i.e. data dependent.)

In problems such as density estimation or optimal transport, one often defines an optimization over a function space F . However, the appropriate specification of F , particularly its smoothness or complexity, should depend on the sample points $\{x_i\}$ and $\{y_j\}$ themselves.

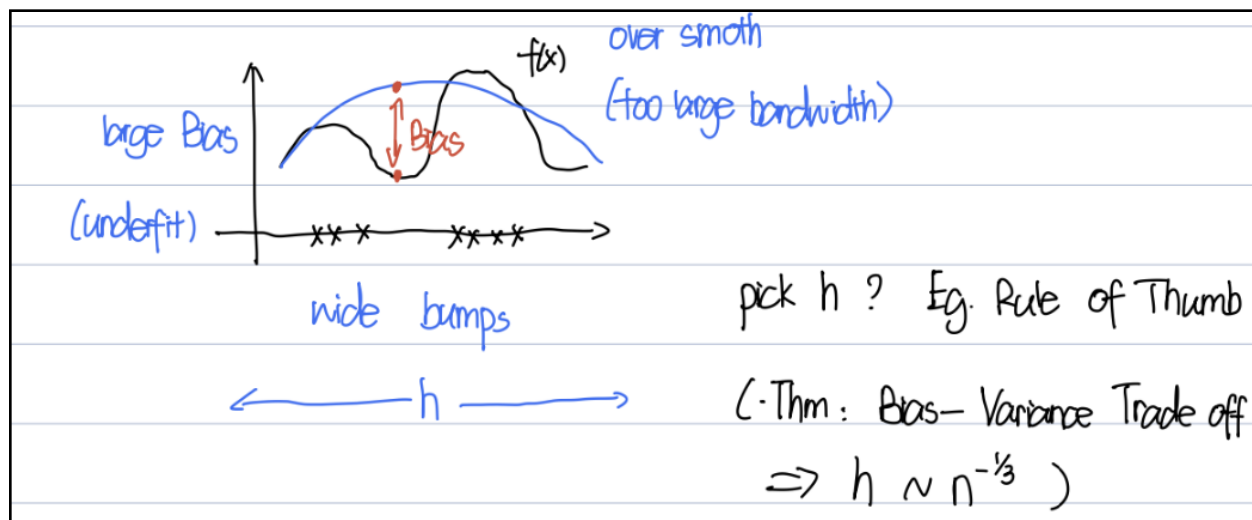
The central idea is that one should allow more flexibility in regions where data is dense, while enforcing stronger smoothness constraints in regions where data is sparse. In other words:

- In regions with many samples, a richer class of functions can be used to capture local structure.
- In regions with few samples, the function space should be restricted to prevent overfitting and ensure robustness.

There are two primary strategies to achieve this adaptivity:

- **Basis Selection:** Choose a basis for F that enforces smoothness directly by controlling the frequency or spatial scale of basis functions. This approach is much like using a large bandwidth in kernel density estimation.
- **Penalization:** Introduce a regularization term that penalizes non-smooth functions. This penalty can be made data-dependent to reflect the local sample density.

In either case, the choice of F must be adaptive to the data, ensuring that the learned function balances expressiveness and generalization.



A Flow-Based, Primal-Dual Approach

We propose a methodology that, rather than solving the primal or dual problems in isolation, combines them through a flow in feature-space (see [1] for a different fluid-based formulation of the optimal transport problem.) Even though our methodology applies to a broad class of cost functions $c(x, y)$, we restrict our attention here for concreteness to the classical cost $c = \frac{1}{2}\|x - y\|^2$, and redefine u and v accordingly as above (The extension to a broader class of convex cost functions is discussed briefly in section 5). In this case the map $y(x)$ that solves Monge's problem can be written in terms of the solution to the dual Kantorovich formulation as

$$y(x) = \nabla u(x).$$

The Link Between Monge and Kantorovich

Rather than solving the primal or dual optimal transport problems in isolation, we adopt a methodology that combines them through a continuous flow in feature space.

Although this approach is applicable to a wide class of cost functions $c(x, y)$, for concreteness we focus here on the classical quadratic cost:

$$c(x, y) = \frac{1}{2} \|x - y\|^2.$$

We are told:

$$y(x) = \nabla u(x),$$

where $u(x)$ is the Kantorovich potential from the dual formulation.

This result forms a deep connection between the **Monge formulation** and the **Kantorovich dual problem**:

- The Monge formulation seeks a deterministic transport map $T(x)$ that pushes the source distribution $\rho(x)$ onto the target $\mu(y)$, minimizing the transport cost.
- The Kantorovich formulation relaxes this by allowing probabilistic couplings and introduces dual potentials $u(x)$, $v(y)$ satisfying the inequality $u(x) + v(y) \leq c(x, y)$.

This result bridges these perspectives: when the cost is quadratic and the source measure is absolutely continuous, the optimal map from Monge’s formulation is precisely the gradient of the potential function that solves the Kantorovich dual problem. This shows that Monge’s transport map is not arbitrary—it arises from the geometry of convex optimization.

The reason the optimal map takes the form $y(x) = \nabla u(x)$ rather than involving $v(y)$ directly lies in the structure of the dual problem. The function $u(x)$ is convex by construction, because the dual constraint $u(x) + v(y) \leq c(x, y)$ implies convexity when $c(x, y)$ is quadratic and symmetric in x . The gradient $\nabla u(x)$ thus defines a direction of steepest ascent that naturally yields the optimal transport destination for each point x .

Although both $u(x)$ and $v(y)$ are solutions to the dual problem, the Monge map is recovered specifically from the x -side potential $u(x)$, since we are pushing forward from x to y . The function $v(y)$ plays a supporting role in enforcing dual feasibility and encoding optimality conditions from the y -side.

In fact, $v(y)$ can be expressed as the Legendre–Fenchel transform (also called the c -transform) of $u(x)$. For quadratic cost, this takes the form:

$$v(y) = \inf_{x \in \mathbb{R}^d} [c(x, y) - u(x)] = \inf_{x \in \mathbb{R}^d} \left[\frac{1}{2} \|x - y\|^2 - u(x) \right],$$

where the infimum is taken over all x in $\mathbb{R}^d$. This formulation guarantees that the inequality $u(x) + v(y) \leq c(x, y)$ holds, with equality on the support of the optimal coupling.

In summary, although both potentials participate in the Kantorovich dual formulation, the optimal transport map from the Monge formulation emerges directly as the gradient of the convex function $u(x)$, not from $v(y)$.

We propose a methodology that, rather than solving the primal or dual problems in isolation, combines them through a flow in feature-space (see [1] for a different fluid-based formulation of the optimal transport problem.) Even though our methodology applies to a broad class of cost functions $c(x, y)$, we restrict our attention here for concreteness to the classical cost $c = \frac{1}{2}\|x - y\|^2$, and redefine u and v accordingly as above (The extension to a broader class of convex cost functions is discussed briefly in section 5). In this case the map $y(x)$ that solves Monge’s problem can be written in terms of the solution to the dual Kantorovich formulation as

$$y(x) = \nabla u(x).$$

We shall consider this map $y(x)$ as the endpoint of a time-dependent flow $z(x, t)$, such that

$$z(x, 0) = x, \quad z(x, \infty) = y(x).$$

Then, rather than minimizing the objective function directly, we compute the map from ρ to μ by means of the following gradient flow:

$$\begin{cases} \dot{z} = -\nabla_z \left[\frac{\delta \tilde{D}_t}{\delta u} \right]_{u=\frac{1}{2}\|x\|^2} \\ z(x, 0) = x \end{cases} \quad (33)$$

where

$$\tilde{D}_t = \int u(z) \rho_t(z) dz + \int u^*(y) \mu(y) dy \quad (34)$$

and ρ_t is the evolving probability distribution underlying the points $z(x, t)$ that are obtained through the flow in (33); in particular, $\rho_0(z) = \rho(z)$ (After considering here the regular transportation problem where $\rho(x)$ and $\mu(y)$ are known,

Static vs. Dynamic: Same Map, Different Paths

There’s no contradiction — both views describe the same transport map:

- **Static view:** The optimal map is $y(x) = \nabla u(x)$, the gradient of a convex function u .
- **Dynamic view:** You start with the identity map and follow a gradient flow. As $t \rightarrow \infty$, the flow converges to $y(x) = \nabla u(x)$.

Gradient Flow vs. Gradient Descent (Discretized Flow)

The notation $\dot{z}(x, t)$ means the time derivative of $z(x, t)$, that is:

$$\dot{z}(x, t) = \frac{\partial z(x, t)}{\partial t}$$

This describes how $z(x, t)$ changes continuously over time under the influence of some gradient.

In practice, we often discretize time into steps $t^n, t^{n+1}, \dots$ with the assumption that $t^{n+1} - t^n = \Delta t$ is constant, and approximate the continuous flow using finite differences:

$$\dot{z}(x, t^n) \approx \frac{z(x, t^{n+1}) - z(x, t^n)}{\Delta t}$$

$$\frac{dy}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

$$m = \lim_{x \rightarrow a} \frac{f(x) - f(a)}{x - a}$$

Calcworkshop.com

Multiplying both sides by Δt , we get:

$$z(x, t^{n+1}) - z(x, t^n) \approx \dot{z}(x, t^n) \cdot \Delta t$$

$$\Rightarrow z(x, t^{n+1}) \approx z(x, t^n) + \Delta t \cdot \dot{z}(x, t^n)$$

Now, if the flow is defined by moving along the negative gradient of some functional (e.g., $\dot{z}(x, t) = -\nabla_z F(z(x, t))$), then this becomes:

$$z(x, t^{n+1}) = z(x, t^n) - \eta \cdot \nabla_z F(z(x, t^n))$$

This is exactly the gradient descent update rule, where $\eta = \Delta t$ is the learning rate (i.e., step size), and F is the objective we're minimizing.

Derivation of Equation (34): Time-Dependent Dual Functional

For a cost function $c(x, y)$, the Kantorovich dual problem is given by:

$$\sup_{u, v} \left\{ \int u(x) \rho(x) dx + \int v(y) \mu(y) dy \right\} \quad \text{subject to } u(x) + v(y) \leq c(x, y)$$

Let $c(x, y) = \frac{1}{2} \|x - y\|^2$. At optimality, the constraint becomes tight:

$$u(x) + v(y) = \frac{1}{2} \|x - y\|^2$$

Solving for $v(y)$, we obtain:

$$v(y) = \inf_x \left\{ \frac{1}{2} \|x - y\|^2 - u(x) \right\} = u^*(y),$$

which is the Legendre–Fenchel transform of $u(x)$.

Substituting $v(y) = u^*(y)$ into the dual objective gives:

$$D(u) = \int u(x) \rho(x) dx + \int u^*(y) \mu(y) dy$$

Let $z(x, t)$ be a time-dependent map with:

$$z(x, 0) = x, \quad z(x, \infty) = y(x)$$

Let $\rho_t(z)$ denote the pushforward of $\rho(x)$ under $z(x, t)$, i.e.,

$$\rho_t = z(x, t)_\# \rho(x)$$

In this context, x represents **samples from the source distribution** $\rho(x)$, not the target μ .

Here's how you know:

- The formula:

$$\tilde{\rho}(x) = \mu(T(x)) \cdot \left| \det \left(\frac{\partial T}{\partial x}(x) \right) \right|$$

is the **forward-mode change-of-variables formula**, where you're starting with samples $x \sim \rho$, pushing them through the map T , and expressing the density of the pushforward $T_\# \rho = \mu$.

$$\rho(x) = \rho_t(z(x, t)) \cdot \left| \det \left(\frac{\partial z}{\partial x}(x, t) \right) \right|$$

$$\rho_t(z(x, t)) = \rho(x) \cdot \left| \det \left(\frac{\partial z}{\partial x}(x, t) \right) \right|^{-1}$$

Replacing $\rho(x)$ with the time-evolving $\rho_t(z)$, we obtain:

$$\tilde{D}_t(u) = \int u(z) \rho_t(z) dz + \int u^*(y) \mu(y) dy$$

This is Equation (34). It reflects the dual energy at time t , where the first term depends on the transported mass and the second term remains fixed.

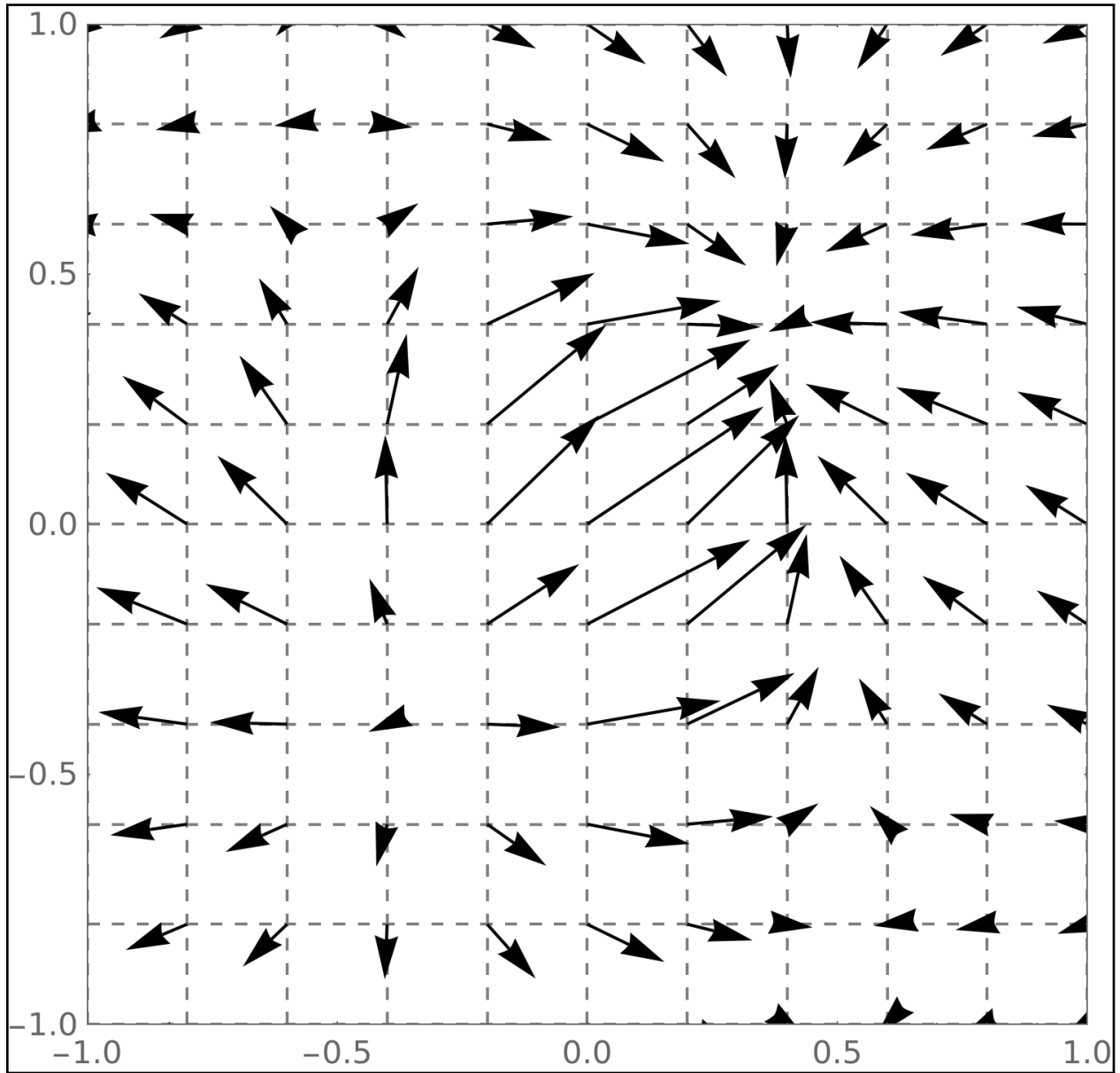
Rather than solving the dual problem directly, we evolve the transport map $z(x, t)$ by flowing in the negative gradient direction of the functional derivative:

$$\dot{z}(x, t) = -\nabla_z \left[\frac{\delta \tilde{D}_t}{\delta u} \right], \quad z(x, 0) = x$$

This defines a gradient flow in feature space that implicitly reduces $\tilde{D}_t(u)$ over time and converges to the optimal transport map $y(x) = \nabla u(x)$.

- This gives you a field over space (at each location z) telling you which direction to push the particle to decrease the energy $\tilde{D}_t$,

Importantly, the vector field guiding the flow evolves over time, since it depends on the energy functional $\tilde{D}_t(u)$, which itself depends on the time-varying density ρ_t . As the particles are transported by the flow $z(x, t)$, the density $\rho_t = z(x, t)_\# \rho(x)$ changes, which in turn updates the vector field used to move the particles. This creates a dynamic feedback loop: the transport field reshapes the distribution, and the evolving distribution continuously reshapes the transport field.



Interpreting the Gradient Flow and Update Rule

The gradient

$$\nabla_z \left[\frac{\delta \tilde{D}_t}{\delta u} \right]$$

is a vector field that assigns a vector to each position vector z .

The update

$$z(x, t^{n+1}) = z(x, t^n) - \eta \cdot \nabla_z F(z(x, t^n))$$

adds that vector (scaled by $-\eta$) to the current position vector z .

Interpretation of the Gradient Flow

We now explain in detail the mechanics and purpose of the gradient flow described in Equation (33):

$$\begin{cases} \dot{z}(x, t) = -\nabla_z \left[\frac{\delta \tilde{D}_t}{\delta u} \right]_{u=\frac{1}{2}\|x\|^2}, \\ z(x, 0) = x, \end{cases}$$

where the dual functional $\tilde{D}_t$ is defined as:

$$\tilde{D}_t = \int u(z) \rho_t(z) dz + \int u^*(y) \mu(y) dy.$$

What Is Evolving Over Time?

The primary object evolving in time is the map $z(x, t)$, which transports samples from the source distribution ρ toward the target distribution μ . The evolving density ρ_t is defined as the pushforward of ρ by the current map $z(x, t)$, i.e.,

$$\rho_t = (z(x, t))_{\#} \rho(x).$$

Thus, the distribution ρ_t changes over time as the flow $z(x, t)$ evolves.

Is the Potential u Changing Over Time?

No. In this formulation, the function u is **fixed** throughout the evolution. Specifically, it is chosen to be the quadratic potential:

$$u(z) = \frac{1}{2} \|z\|^2,$$

which corresponds to the classical quadratic cost $c(x, y) = \frac{1}{2} \|x - y\|^2$. This function defines the energy landscape over which the particles are transported, and it is not optimized or updated during the flow.

Why Take the Functional Derivative $\delta \tilde{D}_t / \delta u$ If u Is Fixed?

Although u itself is fixed, the functional $\tilde{D}_t$ depends on the evolving density ρ_t , which in turn depends on the evolving particle configuration $z(x, t)$. The gradient of $\tilde{D}_t$ with respect to u , evaluated at the fixed potential $u = \frac{1}{2} \|z\|^2$, yields a vector field over z that indicates the direction in which the current particle positions should be moved in order to decrease the value of $\tilde{D}_t$.

This is not optimization over u ; instead, we are using the derivative of $\tilde{D}_t$ as a guide for how to update $z(x, t)$. In this way, the flow performs gradient descent on the energy landscape defined by the fixed u , evolving the mass configuration toward the optimal transport map.

we shall translate these ideas to our case of interest, where only samples from both are given.)

The variational derivative of $\tilde{D}_t$ adopts the form

$$\frac{\delta \tilde{D}_t}{\delta u} = \rho_t - \mu(\nabla u^{**}) \det(D^2 u^{**}). \quad (35)$$

Mesh-Free Implementation

In this section, we adapt the flow above to applications where the densities $\rho(x)$ and $\mu(y)$ are known only through samples $x_i, i \in [1 \dots m]$ and $y_j, j \in [1 \dots n]$ respectively. The following notation will be used:

- $z^k(x)$ is the map at the k -th iteration step, with $z^0(x) = x$ and $z_i^k = z^k(x_i)$.

- $\rho^k(z)$ is the density obtained by mapping $\rho(x)$ via the map $z^k(x)$, and

$$\hat{D}^k \equiv \frac{1}{m} \sum_{i=1}^m u(z_i^k) + \frac{1}{n} \sum_{j=1}^n u^*(y_j) \quad (41)$$

is the Monte Carlo estimate of the functional

$$\tilde{D}^k(u) = \int_{K_1} u(x) \rho^k(z) dz + \int_{K_2} u^*(y) \mu(y) dy. \quad (42)$$

In the mesh-free version of the algorithm, rather than following the gradient of $\frac{\delta \tilde{D}_t}{\delta u}$, we restrict the maps at each step to a family of perturbations to the identity: $z_i^{k+1} = \nabla u_\beta^k(z_i^k)$, where $u_\beta^k(z) = \frac{z^2}{2} + \sum_l \beta_l F_l^k(z)$, with $F_l^k(z)$ a given set of functions and β a vector of free parameters. At each step k , we perform the following operations:

Explanation of the Mesh-Free Update Rule

In the mesh-free version of the algorithm, we avoid computing the full functional gradient

$$\frac{\delta \tilde{D}_t}{\delta u}$$

over the entire space, which would be computationally expensive. Instead, we restrict our attention to a simpler family of maps that are **perturbations of the identity map**.

At each step k , the transport map is updated using:

$$z_i^{k+1} = \nabla u_\beta^k(z_i^k)$$

where the function $u_\beta^k(z)$ is defined as:

$$u_\beta^k(z) = \frac{1}{2} \|z\|^2 + \sum_l \beta_l F_l^k(z)$$

Here:

- The term $\frac{1}{2} \|z\|^2$ corresponds to the identity map, since its gradient is $\nabla \left(\frac{1}{2} \|z\|^2 \right) = z$.

- The remaining term $\sum_l \beta_l F_l^k(z)$ is a perturbation — a linear combination of basis functions $F_l^k(z)$, weighted by parameters β_l .

Thus, the overall update takes the form:

$$z^{k+1} = z^k + \nabla \left(\sum_l \beta_l F_l^k(z^k) \right) = z^k + \sum_l \beta_l \nabla F_l^k(z^k)$$

Rather than optimizing over all possible functions, we restrict the update to this structured form and optimize over the finite-dimensional parameter vector β . This provides a tractable approximation of the transport map, which is especially suitable in mesh-free settings.

Expanding this recursion yields:

$$z^{k+1}(x) = x + \sum_{i=1}^k \beta^i \nabla F(z^i),$$

which defines a transformation of the form:

$$T(x) := z^{k+1}(x).$$

Convexity Considerations

The transport map $T(x)$ is intended to approximate the gradient of a convex potential function. Our construction achieves this under the following conditions:

- **Positive weights:** Convex functions are closed under positive linear combinations. Since the gradient of a sum of convex functions (with positive weights) is equivalent to the sum of their gradients, the resulting transformation remains the gradient of a convex function.
- **Negative weights:** If any $\beta^i < 0$, convexity may be lost. In this case, the sum of gradients may no longer correspond to the gradient of a convex function. Careful control of step sizes is therefore necessary to preserve the desired structure.
- **Sampled gradients:** In practice, the gradients $\nabla F(z^i)$ are computed only at a finite collection of sample points z^i . The map $T(x)$ is therefore constructed by stitching together directional updates at these discrete locations. However, Brenier's theorem requires the overall transformation to behave like the gradient of a single globally defined convex function. As such, the sum

$$\sum_{i=1}^k \beta^i \nabla F(z^i)$$

must approximate—at least locally—the gradient of a convex potential. This imposes structural constraints on the sampling strategy, the choice of weights β^i , and the properties of the function F itself.

Interpretation and Goal

This iterative construction defines a flow of the point x under successive applications of directional updates determined by ∇F . The final position $z^{k+1}(x)$ defines a map that can be interpreted as a transport map:

$$T(x) = z^{k+1}(x),$$

provided it satisfies the convexity condition.

What is convex?

The convexity consideration applies to the **potential function** $u(z)$ whose gradient defines the transport map. Specifically, we aim for the transport map

$$T(x) := \nabla u(z)$$

to be the gradient of a convex function.

For example, suppose the potential function is given by:

$$u(z) = \frac{1}{2} \|z\|^2 + \sum_i \beta^i F^i(z),$$

then:

- If each $F^i(z)$ is convex and all $\beta^i \geq 0$, then $u(z)$ is convex.
- In that case, the map $T(x) = \nabla u(z)$ is a valid Brenier map (i.e., the gradient of a convex function).

Subscript l : Basis function index

- $l \in \{1, 2, \dots, L\}$ indexes **which basis function** you're using.
- You're typically summing over l in expressions like:

$$\sum_l \beta_l F_l^k(z)$$

- So this tells you you're using a finite **set of basis functions** $\{F_1^k, F_2^k, \dots\}$.

Superscript k : Iteration number

- k denotes the **time step or iteration number** in the algorithm.
- These basis functions F_l^k may **change at each step k** , for example if they are computed using samples z^k , or if their parameters are recomputed each iteration.

Together:

- $F_l^k(z)$ means:
 - ▮ The l -th basis function used at iteration k , evaluated at point z .

What's going on:

At iteration $k = 0$, you start with:

$$z^0(x) = x$$

Then the updates look like:

$$z^1(x) = \nabla u_\beta^0(z^0(x)) = x + \text{perturbation}_0$$

$$z^2(x) = \nabla u_\beta^1(z^1(x)) = z^1(x) + \text{perturbation}_1$$

$$z^3(x) = \nabla u_\beta^2(z^2(x)) = z^2(x) + \text{perturbation}_2$$

So what you're really doing is this:

$$z^3(x) = x + \text{pert}_0 + \text{pert}_1 + \text{pert}_2$$

1. Compute a $\bar{\beta}$ that reduces the value of

$$\hat{D}^k(\beta) = \frac{1}{m} \sum_{i=1}^m u_\beta^k(z_i^k) + \frac{1}{n} \sum_{j=1}^n (u_\beta^k)^*(y_j), \quad (43)$$

either through gradient descent:

$$\bar{\beta}_l \propto - \left. \frac{\partial \hat{D}^k(\beta)}{\partial \beta_l} \right|_{\beta=0}, \quad (44)$$

or through Newton's method, as described in section (4.1).

Derivation of the Gradient Descent Update for $\bar{\beta}_l$

We aim to minimize the sampled dual objective:

$$\hat{D}^k(\beta) = \frac{1}{m} \sum_{i=1}^m u_\beta^k(z_i^k) + \frac{1}{n} \sum_{j=1}^n (u_\beta^k)^*(y_j),$$

where the convex potential $u_\beta^k(z)$ is defined as:

$$u_\beta^k(z) = \frac{1}{2} \|z\|^2 + \sum_{l=1}^L \beta_l F_l^k(z).$$

To perform gradient descent, we compute the derivative of $\hat{D}^k(\beta)$ with respect to a single coefficient β_l :

$$\frac{\partial \hat{D}^k(\beta)}{\partial \beta_l} = \frac{1}{m} \sum_{i=1}^m \frac{\partial u_{\beta}^k(z_i^k)}{\partial \beta_l} + \frac{1}{n} \sum_{j=1}^n \frac{\partial}{\partial \beta_l} (u_{\beta}^k)^*(y_j).$$

Unrolling the Derivative Step-by-Step

We are minimizing the sampled dual objective:

$$\hat{D}^k(\beta) = \frac{1}{m} \sum_{i=1}^m u_{\beta}^k(z_i^k) + \frac{1}{n} \sum_{j=1}^n (u_{\beta}^k)^*(y_j),$$

with respect to the parameter β_l . To compute the gradient descent update, we take the derivative of this function with respect to β_l .

Step 1: Apply linearity of the derivative to the sum

$$\frac{\partial}{\partial \beta_l} \hat{D}^k(\beta) = \frac{\partial}{\partial \beta_l} \left(\frac{1}{m} \sum_{i=1}^m u_{\beta}^k(z_i^k) \right) + \frac{\partial}{\partial \beta_l} \left(\frac{1}{n} \sum_{j=1}^n (u_{\beta}^k)^*(y_j) \right)$$

Step 2: Pull constants out of the derivative (linearity of scalar multiplication)

$$= \frac{1}{m} \sum_{i=1}^m \frac{\partial u_{\beta}^k(z_i^k)}{\partial \beta_l} + \frac{1}{n} \sum_{j=1}^n \frac{\partial}{\partial \beta_l} (u_{\beta}^k)^*(y_j)$$

At this point, we now compute the two terms individually.

Now we evaluate each derivative separately:

- Since $u_{\beta}^k(z)$ is linear in β_l , we have:

$$\frac{\partial u_{\beta}^k(z_i^k)}{\partial \beta_l} = F_l^k(z_i^k).$$

- For the Legendre dual, we use the known identity:

$$\frac{d}{d\beta} u^*(y) = -\frac{d}{d\beta} u(x), \quad \text{where } y = \nabla u(x).$$

Thus,

$$\frac{\partial}{\partial \beta_l} (u_{\beta}^k)^*(y_j) = -F_l^k(y_j).$$

Combining both terms:

$$\frac{\partial \hat{D}^k(\beta)}{\partial \beta_l} = \frac{1}{m} \sum_{i=1}^m F_l^k(z_i^k) - \frac{1}{n} \sum_{j=1}^n F_l^k(y_j).$$

Gradient descent update: We now apply one step of gradient descent:

$$\bar{\beta}_l \leftarrow \bar{\beta}_l - \eta \cdot \frac{\partial \hat{D}^k(\beta)}{\partial \beta_l}$$

Substituting the expression above:

$$\bar{\beta}_l \leftarrow \bar{\beta}_l - \eta \left(\frac{1}{m} \sum_{i=1}^m F_l^k(z_i^k) - \frac{1}{n} \sum_{j=1}^n F_l^k(y_j) \right).$$

Gradient Descent Update Forms

Vector form (full gradient update):

$$\beta \leftarrow \beta - \eta \cdot \nabla_{\beta} \hat{D}^k(\beta)$$

This updates *all* components β_l at once, using the full gradient vector.

Component-wise form (per-coordinate update):

$$\bar{\beta}_l \leftarrow \bar{\beta}_l - \eta \cdot \frac{\partial \hat{D}^k(\beta)}{\partial \beta_l}$$

This is still gradient descent — you’re just showing the update for *one component* at a time.

The full gradient is the vector of all partial derivatives:

$$\nabla_{\beta} \hat{D}^k(\beta) = \begin{bmatrix} \frac{\partial \hat{D}^k}{\partial \beta_1} \\ \frac{\partial \hat{D}^k}{\partial \beta_2} \\ \vdots \\ \frac{\partial \hat{D}^k}{\partial \beta_L} \end{bmatrix}$$

Updating each component individually using its partial derivative is equivalent to applying the full gradient update when done simultaneously.

Interpreting the Gradient: Direction, Steepness, and Step Size

$$\beta \leftarrow \beta - \eta \nabla f(\beta)$$

The gradient vector $\nabla f(\beta)$ contains two key pieces of information:

- The **direction** points toward the steepest increase of the function f ,
- The **magnitude** tells you how steep that increase is.

Since we’re minimizing, we move in the opposite direction $-\nabla f(\beta)$. But how far should we go?

The step size η acts as a **trust factor**. It controls how far we’re willing to follow the gradient’s advice.

Analogy: Think of the gradient as your GPS giving you directions based on a local map. The vector tells you which direction to go, and how steep the slope is — but the GPS only knows the terrain right around your current location. It doesn’t know what’s coming just a bit further out.

The step size η is how far you’re willing to follow that local advice before stopping to recheck. A small η means you’re being cautious — trusting the local gradient only a little. A large η means you’re taking a longer stride based on the current slope.

In essence: the gradient tells you what the terrain looks like *locally*, and η controls how far you extrapolate that information to update your position.

2. Update the sample points,

$$z_i^{k+1} = \nabla u_{\tilde{\beta}}^k(z_i^k) \quad (45)$$

as well as any other points z_s^k where the map is sought. If the density $\rho^0(x)$ is known or estimated, as is the case in some applications and in synthetic examples, $\rho^{k+1}(z)$ can also be updated using

$$\rho^k(z) = J^{\nabla u_{\tilde{\beta}}^k}(z) \rho^{k+1}(\nabla u_{\tilde{\beta}}^k(z)), \quad (46)$$

where $J^{\nabla u_{\tilde{\beta}}^k}(z)$ is the Jacobian of the map $\nabla u_{\tilde{\beta}}^k$.

The algorithm converges when $\hat{D}^k$ reaches a minimum, and so $\beta \rightarrow 0$. Since the derivative of $\hat{D}^k$ with respect to β_l is

$$\left. \frac{\partial \hat{D}^k}{\partial \beta_l} \right|_{\beta=0} = \frac{1}{m} \sum_{i=1}^m F_l^k(z_i^k) - \frac{1}{n} \sum_{j=1}^n F_l^k(y_j), \quad (47)$$

then at convergence we have that

$$\frac{1}{m} \sum_{i=1}^m F_l^\infty(z_i^\infty) = \frac{1}{n} \sum_{j=1}^n F_l^\infty(y_j) : \quad (48)$$

In gradient descent, you stop updating when the gradient is zero. That's your stopping condition:

$$\frac{\partial \hat{D}^k}{\partial \beta_l} = 0 \quad \text{for all } l.$$

the mapped sample points z_i^∞ are such that they provide the same Monte Carlo estimates as the samples y_j from $\mu(y)$ for the expected values of all candidate functions F_l^∞ . This is our sample-driven, weak formulation of the transport condition $\rho^\infty(y) = \mu(y)$. Its level of resolution depends on the richness of the family F from where the F_l^k are chosen, with too rich or too small a family yielding over and under resolution respectively. Candidate families F are discussed below.

Map-Based Perspective

In this view, the objective is to compute the optimal transport map:

$$y(x) = \nabla u(x)$$

This map is the solution to Monge's problem and is defined as the endpoint of a flow:

$$z(x, 0) = x, \quad z(x, \infty) = y(x)$$

The goal is to recover this map $y(x)$ explicitly.

Distribution-Based Perspective

Alternatively, we can view the process through the lens of distribution matching. If we start with samples $x_i \sim \rho$ from the source distribution and evolve them via the flow $z(x, t)$, then the resulting pushed-forward samples $z(x_i, t)$ define a time-dependent density ρ_t . The goal is to ensure:

$$\rho_t \xrightarrow{t \rightarrow \infty} \mu$$

In other words, the distribution of the transformed samples should converge to the target distribution μ .

Connecting the Flow Map and Distributional Convergence

These two formulations are equivalent. We consider a time-dependent flow $z(x, t)$ that evolves particles starting from $x \sim \rho$. The flow satisfies:

$$z(x, 0) = x, \quad z(x, t) \xrightarrow{t \rightarrow \infty} y(x) = \nabla u(x),$$

where $y(x)$ is the optimal transport map. As $t \rightarrow \infty$, the flow map converges to $y(x)$, so the induced particle distribution converges as well:

$$\rho_t \xrightarrow{t \rightarrow \infty} (y(x))_{\#} \rho = \mu.$$

Introduce an Auxiliary Density and Pushforward Map

We define a new density $\tilde{\rho}$ on $\mathbb{R}^d$ and a smooth, invertible map $T : \mathbb{R}^d \rightarrow \mathbb{R}^d$ such that:

$$\mu = T_{\#} \tilde{\rho}$$

Here, $T_{\#} \tilde{\rho}$ denotes the *pushforward* of $\tilde{\rho}$ through T , i.e., the distribution of $T(x)$ when $x \sim \tilde{\rho}$. This construction allows us to represent the known reference density μ in terms of an unknown $\tilde{\rho}$ and a transformation T .

There are two distinct but mathematically connected perspectives on how to construct a transport map:

Flow-Based View (Sample-Driven, Dynamical)

- **Setup:** You are given a set of samples from a known source distribution ρ , and you want to construct a dynamical process that evolves those points toward a target distribution μ .
- **Construction:** Define a time-dependent flow $z(x, t)$ that moves particles over time.

$$z(x, 0) = x, \quad \lim_{t \rightarrow \infty} z(x, t) = y(x)$$

where $y(x) = \nabla u(x)$ is the final transport map.

- **Pushforward Condition:**

$$\rho_t = (z(x, t))_{\#} \rho \quad \text{and} \quad \rho_t \xrightarrow{t \rightarrow \infty} \mu$$

So ρ is evolved via the flow to approximate μ .

Pushforward View (Change of Variables, Density-Based)

- **Setup:** You start with a known base distribution $\tilde{\rho}$ (e.g., a standard Gaussian), and you want to construct a map $T(x)$ that pushes this density to the target μ .
- **Construction:** Use change-of-variables:

$$\mu = T_{\#}\tilde{\rho}$$

which means the distribution of $T(x)$ when $x \sim \tilde{\rho}$ is exactly μ .

- **Interpretation:** This is the standard framework for normalizing flows and variational inference: start from a simple density and learn a transformation T that models the target.

Key Distinction

- The **flow-based view** is *sample-driven*: you evolve particles drawn from ρ using a flow $z(x, t)$ toward μ .
- The **pushforward view** is *density-driven*: you apply a smooth map T to a known density $\tilde{\rho}$ to recover μ .

Computing β

Because of the simplicity of each elementary map, one can choose β at each time step so as to minimize the local quadratic approximation to $\hat{D}$

$$\hat{D} \approx \hat{D}_0 + G\beta + \frac{1}{2}\beta^T H \beta \quad (49)$$

where

$$\hat{D}_0 = \hat{D}|_{\beta=0}, \quad G_i = \left. \frac{\partial \hat{D}}{\partial \beta_i} \right|_{\beta=0}, \quad H_{ij} = \left. \frac{\partial^2 \hat{D}}{\partial \beta_i \partial \beta_j} \right|_{\beta=0}. \quad (50)$$

Minimizing (49) with respect to β yields

$$\beta = -H^{-1}G, \quad (51)$$

which one may want to cap by a *learning rate* ϵ to avoid big jumps based only on local information:

$$\beta \rightarrow \alpha\beta, \quad \text{with } \alpha = \min(1, \frac{\epsilon}{\|\beta\|}).$$

Newton Step for Choosing the Flow Parameter β

This section describes a Newton-style step size selection strategy for determining the update direction and magnitude of the flow parameter β , which governs the transport map via elementary flows.

Goal

The goal is to choose β at each time step so as to **minimize the objective function $\hat{D}(\beta)$** using a **local quadratic approximation**.

Quadratic Approximation (Equation 49)

To this end, the function $\hat{D}(\beta)$ is locally approximated by its second-order Taylor expansion around $\beta = 0$:

$$\hat{D}(\beta) \approx \hat{D}_0 + G^T \beta + \frac{1}{2} \beta^T H \beta,$$

where:

- $\hat{D}_0 = \hat{D}|_{\beta=0}$ is the value of the function at the current step.
- $G \in \mathbb{R}^d$ is the gradient vector with components

$$G_i = \left. \frac{\partial \hat{D}}{\partial \beta_i} \right|_{\beta=0}.$$

- $H \in \mathbb{R}^{d \times d}$ is the Hessian matrix with entries

$$H_{ij} = \left. \frac{\partial^2 \hat{D}}{\partial \beta_i \partial \beta_j} \right|_{\beta=0}.$$

This gives a local quadratic model of the objective near the current iterate.

Minimizing the Quadratic (Equation 51)

To minimize the quadratic approximation, we solve:

$$\nabla_{\beta} \left(G^T \beta + \frac{1}{2} \beta^T H \beta \right) = 0,$$

which yields the classical Newton update:

$$\beta = -H^{-1}G.$$

This direction not only follows the steepest descent but also adapts to the local curvature of the function (via the Hessian).

Controlling the Step Size

To avoid overly large steps that might be unreliable due to the local nature of the approximation, the step is scaled by a learning rate ϵ :

$$\beta \rightarrow \alpha \beta, \quad \text{with} \quad \alpha = \min \left(1, \frac{\epsilon}{\|\beta\|} \right).$$

This ensures the step remains within a trusted radius, behaving similarly to a *trust region* strategy.

Gradient of a Constant = 0

- Since $\hat{D}_0$ doesn't depend on β , its gradient with respect to β is:

$$\nabla_{\beta} \hat{D}_0 = 0.$$

- So when we compute:

$$\nabla_{\beta} \left(\hat{D}_0 + G^T \beta + \frac{1}{2} \beta^T H \beta \right),$$

we only keep the terms involving β , because $\hat{D}_0$ drops out.

The gradient G of $\hat{D}(\beta)$ has already been computed in (47). Since the Hessian of the first sum in (43) is zero, we only need to compute the Hessian of the second sum, involving the Legendre transform of F . Introducing

$$\bar{z}(y, \beta) = \arg \max_z \left[z \cdot y - \frac{\|z\|^2}{2} - \sum_l \beta_l F_l(z) \right] = y - \sum_l \beta_l \nabla F_l(\bar{z}), \quad (52)$$

we have that

$$\begin{aligned} \left(\frac{\|z\|^2}{2} + \sum_l \beta_l F_l(z) \right)^* (y) &= \max_z \left[z \cdot y - \frac{\|z\|^2}{2} - \sum_l \beta_l F_l(z) \right] \quad (53) \\ &= \bar{z} \cdot y - \frac{\|\bar{z}\|^2}{2} - \sum_l \beta_l F_l(\bar{z}) = \\ &= \frac{\|y\|^2}{2} - \sum_l \beta_l F_l(y) + \frac{1}{2} \sum_{i,j} \beta_i \beta_j \nabla F_i(y) \cdot \nabla F_j(y) + O(\|\beta\|^3), \end{aligned}$$

so

$$G_l = \frac{1}{m} \sum_{i=1}^m F_l(z_i) - \frac{1}{n} \sum_{j=1}^n F_l(y_j) \quad (54)$$

and

$$H_{ls} = \frac{1}{n} \sum_{j=1}^n \nabla F_l(y_j) \cdot \nabla F_s(y_j). \quad (55)$$

This section uses the **Legendre transform** and a **Taylor expansion** to compute the gradient and Hessian of the objective function $\hat{D}(\beta)$. These expressions are then plugged into the **Newton update step** to compute the optimal parameter update β at each iteration.

The Family of Potentials F

In order to complete the description of the procedure above, we need to specify the form and number of the functions $F_l(z)$ to use at each time-step. The following considerations apply:

1. In order neither to over-resolve nor to under-resolve the probability densities $\rho^k(z)$ and $\mu(y)$ underlying the samples $\{z_i^k\}$ and $\{y_j\}$, the functions F_l must have length scales consistent with these densities.
2. The function F_l should be tailored so as to work well in high-dimensional settings.

An example of elementary potential satisfying the characteristic stated above is the radially symmetric

$$F_l(z) = r \operatorname{erf}\left(\frac{r}{\alpha}\right) + \frac{\alpha}{\sqrt{\pi}} e^{-\left(\frac{r}{\alpha}\right)^2}, \quad (56)$$

where $r = |z - \tilde{z}_l|$, and α is a bandwidth dependent on the center $\tilde{z}_l$:

$$\alpha \propto \left(\frac{n_p}{n+m} \left(\frac{1}{\tilde{\rho}(\tilde{z}_l)} + \frac{1}{\tilde{\mu}(\tilde{z}_l)} \right) \right)^{\frac{1}{d}}.$$

Reference: Pages 11-13 in Polymath Jr Tanya Jul 12, 2025 to understand the Kernel Density Estimator.

The bandwidth parameter α determines how wide or narrow the radially symmetric bump function is, and it is computed adaptively according to local density information. The formula is given by:

$$\alpha \propto \left(\frac{n_p}{n+m} \left(\frac{1}{\tilde{\rho}(\tilde{z}_\ell)} + \frac{1}{\tilde{\mu}(\tilde{z}_\ell)} \right) \right)^{1/d}$$

Explanation of Terms

- $\tilde{z}_\ell$: the center of the radial potential F_ℓ
- $\tilde{\rho}(\tilde{z}_\ell), \tilde{\mu}(\tilde{z}_\ell)$: kernel density estimates of the source and target densities at $\tilde{z}_\ell$
- n_p : the number of particles we want this potential to influence
- n, m : the total number of samples from the source and target distributions
- d : the ambient dimension

Intuition Behind Density Adaptivity

The term

$$\left(\frac{1}{\tilde{\rho}(\tilde{z}_\ell)} + \frac{1}{\tilde{\mu}(\tilde{z}_\ell)} \right)$$

is large when both estimated densities are small — i.e., when the region is **sparse**.

- In sparse regions, we want the potential F_ℓ to have **larger spatial influence** to ensure it captures enough particles. A larger α leads to a wider, more spread-out bump.
- In dense regions, many particles are already nearby. So we want F_ℓ to be **tightly concentrated**, pushing only the immediate local mass. A smaller α results in a narrower bump.

This adaptivity helps ensure that each elementary flow affects roughly the same amount of probability mass, regardless of local density variations.

Interpretation of the Scaling Factor $\left(\frac{n_p}{n+m} \right)^{1/d}$

The factor

$$\left(\frac{n_p}{n+m} \right)^{1/d}$$

rescales the bandwidth to ensure that, in expectation, each potential F_ℓ influences approximately n_p out of the total $n + m$ points.

More specifically:

- $\frac{n_p}{n+m}$ represents the *fraction* of total points that we want to fall within the support of F_ℓ .
- Raising this quantity to the power $\frac{1}{d}$ transforms the fraction of *volume* into a corresponding *length scale*, appropriate for d -dimensional Euclidean space.

This dimensional scaling is important: in high dimensions, small changes in radius result in large changes in volume, so the adjustment ensures consistent influence across different dimensions.

Do Only Nearby Points Get Pushed by the Gradient Update?

- The gradient $\nabla F_\ell(z)$ is nonzero everywhere, but its strength depends on the distance from the center $\tilde{z}_\ell$.
- It grows rapidly near $\tilde{z}_\ell$, then flattens as $\|z - \tilde{z}_\ell\| \gg \alpha$.
- Faraway points receive nearly constant, directionless pushes — effectively contributing nothing to transport.
- Thus, although the field has global support, it acts locally: only points within distance $\sim \alpha$ are meaningfully updated.

Here n_p is the desired number of points to be “captured” by F_l , and $\tilde{\rho}$ and $\tilde{\mu}$ are rough approximations to ρ^k and μ . The elementary map associated with this proposal is given by

$$z^{k+1} = z^k + \sum_l \beta_l f_l(r)(z^k - \tilde{z}_l) \quad (57)$$

where

$$f(r) = \frac{1}{r} \frac{dF}{dr} = \frac{\text{erf}(r/\alpha)}{r}. \quad (58)$$

Reference: Pages 8 & 9 in Polymath Jr Giulio 07-25-2025 and the boxed equation on page 23 of this document.

Radial Function Definition

We consider scalar potentials $F(x)$ of the form:

$$F(x) = \Phi(\|x - \bar{x}\|),$$

where:

- $\bar{x} \in \mathbb{R}^d$ is a fixed center point (e.g., sampled from the source distribution ρ),
- $r = \|x - \bar{x}\|$ is the radial distance from x to $\bar{x}$,
- $\Phi : \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}$ is a smooth convex scalar function.
- $F(x)$ is a radially symmetric and convex function.

Why $F(x)$ and Φ Do Not Share the Same Properties

$F(x)$ and Φ are **not** equal - they're related by composition. So $F(x)$ inherits:

- **Convexity** from both Φ being convex and $\|x - \bar{x}\|$ being convex
- **Radial symmetry** by construction
- But **not necessarily smoothness** due to potential issues at the center point

For example, $\Phi(r) = r$ is smooth, but the resulting $F(x) = \|x - \bar{x}\|$ is not smooth at $x = \bar{x}$ because the gradient of $\|x - \bar{x}\|$ is $\frac{x - \bar{x}}{\|x - \bar{x}\|}$, but this expression is undefined (involves division by zero) when $x = \bar{x}$.

Differentiating the Radially Symmetric Potential

We are given the function:

$$F_\ell(r) = r \cdot \operatorname{erf}\left(\frac{r}{\alpha}\right) + \frac{\alpha}{\sqrt{\pi}} e^{-\left(\frac{r}{\alpha}\right)^2}$$

We aim to compute the derivative with respect to r , that is,

$$\frac{dF_\ell}{dr}$$

Differentiate the first term

$$\frac{d}{dr} \left(r \cdot \operatorname{erf}\left(\frac{r}{\alpha}\right) \right)$$

Using the product rule:

$$= \operatorname{erf}\left(\frac{r}{\alpha}\right) + r \cdot \frac{d}{dr} \left[\operatorname{erf}\left(\frac{r}{\alpha}\right) \right]$$

Recall the derivative of the error function:

$$\frac{d}{dx} \operatorname{erf}(x) = \frac{2}{\sqrt{\pi}} e^{-x^2} \quad \Rightarrow \quad \frac{d}{dr} \operatorname{erf}\left(\frac{r}{\alpha}\right) = \frac{2}{\alpha\sqrt{\pi}} e^{-(r/\alpha)^2}$$

So the full derivative of the first term is:

$$\operatorname{erf}\left(\frac{r}{\alpha}\right) + \frac{2r}{\alpha\sqrt{\pi}}e^{-(r/\alpha)^2}$$

Differentiate the second term

$$\frac{d}{dr} \left(\frac{\alpha}{\sqrt{\pi}} e^{-(r/\alpha)^2} \right) = \frac{\alpha}{\sqrt{\pi}} \cdot \left(-\frac{2r}{\alpha^2} \right) e^{-(r/\alpha)^2} = -\frac{2r}{\alpha\sqrt{\pi}} e^{-(r/\alpha)^2}$$

Combine the results

The exponential terms from both derivatives cancel:

$$\frac{2r}{\alpha\sqrt{\pi}} e^{-(r/\alpha)^2} - \frac{2r}{\alpha\sqrt{\pi}} e^{-(r/\alpha)^2} = 0$$

So we are left with:

$$\frac{dF_\ell}{dr} = \operatorname{erf}\left(\frac{r}{\alpha}\right)$$

The number of functions $F_l(z)$ to use per time-step is a matter of convenience. One function per step suffices for the algorithm described so far, while two functions is the minimum in the variation below that enforces the map's optimality. Yet more free parameters per step might be desired to accelerate the algorithm's convergence. As for the centers z_l , the simplest choice is to pick them at random from among the $\{z_i^k\}$ and $\{y_j\}$, just making sure that any two centers are not too close to each other. An alternative is to sample more frequently points in areas with low probability density, so as to better resolve the tails of the two distributions.

The rough estimates $\tilde{\rho}$ and $\tilde{\mu}$ used to determine the band-width are computed at time zero, for instance through a simple kernel density estimator. The corresponding $\tilde{\rho}^k(z)$ is then updated multiplying by the Jacobian of the map at each time-step.

Updating the Density Estimate Using the Jacobian

At time $t = 0$, we begin with a set of sample points and use a kernel density estimator (KDE) to construct an initial estimate of the source density:

$$\tilde{\rho}^0(x) \approx \rho(x)$$

As the particles evolve over time according to a smooth flow map $z(x, t)$, the underlying density must also evolve. This evolution is governed by the change-of-variables formula, which expresses how densities transform under smooth mappings:

$$\rho_t(z(x, t)) = \rho(x) \cdot \left| \det \left(\frac{\partial z}{\partial x}(x, t) \right) \right|^{-1}$$

This formula ensures that probability mass is conserved. As the flow map stretches or compresses space, the density adjusts accordingly:

- If the Jacobian determinant is greater than 1, space is expanding and the density decreases.

- If the determinant is less than 1, space is contracting and the density increases.

Therefore, to update our KDE-based density estimate at each time step, we multiply the current estimate by the inverse of the Jacobian determinant of the transport map. This gives a consistent approximation of the evolving density $\tilde{\rho}^k(z)$ as the flow progresses.

A Note on the Effective Supports of ρ and μ

It would be virtually impossible for the functions $F_l(z)$ to resolve simultaneously the $\{z_i\}$ and $\{y_j\}$ if the two densities $\rho^k(z)$ and $\mu(y)$ did not overlap, i.e. if there were regions where one density was significant while the other was negligible or zero. At convergence, $\rho^k \approx \mu$, but there is no reason why the initial $\rho(x)$ could not have an effective support distinct from that of $\mu(y)$.

In order to address this issue, one can pre-process the data-points $\{x_i\}$ so that they lie closer to the $\{y_j\}$, deforming them through an isotropic stretching and rigid translation, without compromising the computation of the optimal map of the original problem. The reason is that the composition of these transformations with optimal maps yields optimal maps. To see this, observe that the composition $h(x) = g \circ s(x)$ of the gradient $g(x) = \nabla_x \phi(x)$ of a convex function ϕ and the linear map $s(x) = ax + b$ can be rewritten as $h(x) = \nabla_x \frac{1}{a} \phi(ax + b)$, where $\frac{1}{a} \phi(ax + b)$ is still a convex function.

More general pre-processing maps not necessarily isotropic, linear or curl-free can be performed, if one then implements the steps described below to enforce the optimality of the final map. A natural choice is to use a first map of the form $y = Ax + b$, with the symmetric matrix A and the vector b chosen so as to transform the empirical mean and covariance of the $\{x_i\}$ into those of the $\{y_j\}$.

If the supports of the source and target distributions $\rho^k(z)$ and $\mu(y)$ are too far apart, then the elementary potentials $F_l(z)$ will struggle to move mass effectively. This happens when the two densities do not significantly overlap (when $\rho^k(z)$ is large in regions where $\mu(y)$ is nearly zero, or vice versa). To fix this, we can *pre-process* the data points $\{x_i\}$ using a simple transformation (stretching or translation) to bring them closer to $\{y_j\}$. This does not compromise the correctness of the transport algorithm, because the composition of an optimal map with certain transformations still yields an optimal map. To make the idea concrete, suppose we start with the convex potential

$$\phi(x) = \frac{1}{2} \|x\|^2,$$

whose gradient is the identity map:

$$g(x) = \nabla \phi(x) = x.$$

Now suppose we first apply an affine transformation to the input:

$$s(x) = 3x + 5.$$

Then the composed map is:

$$h(x) = g(s(x)) = \nabla \phi(3x + 5) = 3x + 5.$$

We now ask: is $h(x)$ still the gradient of some convex function? The answer is yes — define a new potential:

$$\tilde{\phi}(x) = \phi(3x + 5) = \frac{1}{2}\|3x + 5\|^2.$$

Then,

$$\nabla \tilde{\phi}(x) = \nabla_x \left(\frac{1}{2}\|3x + 5\|^2 \right) = 3x + 5 = h(x).$$

Since $\tilde{\phi}$ is convex (being a composition of a convex quadratic with an affine function), this shows that $h(x)$ is still an optimal transport map — it is the gradient of a convex function. In general, composing an optimal map with an affine transformation of the input preserves optimality. This justifies preprocessing the data (e.g., centering or scaling) before computing the transport map. This guarantees that preprocessing the source points with a map of the form $s(x) = ax + b$ will not harm the theoretical optimality of the transport, as long as the final map is expressed as the gradient of a convex function. In practice, this preprocessing helps match the mean and covariance of the source $\{x_i\}$ to those of the target $\{y_j\}$, making the flow easier to learn.

Enforcing the Optimality of the Final Map

As shown in section 3, even though the flow given by (33) maps ρ to μ , it is generally not curl-free, and therefore not optimal. This is accentuated if one pre-conditions the computation with an arbitrary smooth map that brings the $\{x_i\}$ and $\{y_j\}$ closer to each other. Moreover, the numerics are only an approximation to the smooth flow in (33), so even if this should converge to the optimal map, the mesh-free algorithm may not be so precise (not so for the convergence of ρ_t to μ , since this is enforced via the moving samples themselves. The optimality of the map on the other hand depends on their original position, that the algorithm forgets.) In this section, we modify the discrete version of the flow from section 4 so that the resulting map converges to the optimal one.

One possibility, developed in [15] within a grid-based methodology, is based on the results discussed in [2] and [11] stating that any map $w(x)$ from ρ to μ admits a unique decomposition of the form

$$w(x) = \nabla \phi \circ s(x), \tag{59}$$

where ϕ is a scalar convex function and s maps ρ into itself. Therefore, given $w(x)$ one can seek a *rearrangement* map s^{-1} satisfying $w \circ s^{-1} = \nabla \phi$. In our context, one would attempt to rearrange the points $\{w(x_i)\}$ in order to minimize the cost

$$\frac{1}{m} \sum_{i=1}^m |w(x_i) - x_i|^2 \tag{60}$$

while keeping the underlying distribution of $\{w(x_i)\}$ fixed to μ .

Recovering the Optimal Map from a Non-Optimal Transport

Suppose we are given a map $w(x)$ that pushes the source distribution ρ to the target distribution μ , $w_{\#}\rho = \mu$, but this map is not necessarily optimal. In particular, it may not be the gradient of a convex function, and hence it may not minimize the transport cost.

The goal is to **recover the optimal map** $T(x) = \nabla\phi(x)$ from the known but non-optimal map $w(x)$. The key insight, based on **Brenier's Theorem**, is that among all maps that push ρ to μ , the one that minimizes the expected squared distance

$$\mathbb{E} [\|T(x) - x\|^2]$$

is unique and is given by the gradient of a convex function.

Note: In the text, single vertical bars $|\cdot|$ are written in place of proper norm notation $\|\cdot\|$.

Algorithmic Strategy:

- Compute the outputs $\{w(x_i)\}$ for a set of samples $\{x_i \sim \rho\}$.
- Find a **permutation** π that minimizes the total squared cost:

$$\min_{\pi} \frac{1}{m} \sum_{i=1}^m \|w(x_{\pi(i)}) - x_i\|^2$$

- Ensure that the resulting rearranged outputs $\{w(x_{\pi(i)})\}$ have the same empirical distribution as μ .

This cost-minimizing rearrangement ensures that the final map is:

- The closest (in squared distance) to the identity among all maps that push $\rho \rightarrow \mu$,
- And therefore, by Brenier's Theorem, it must be of the form $T(x) = \nabla\phi(x)$ for some convex potential ϕ .

Note: π is a **permutation** of the set $\{1, 2, \dots, m\}$. That is, π is a function that rearranges indices. For example, $\pi(3) = 7$ means you pair x_3 with $w(x_7)$.

Rather than implementing such constrained minimization algorithm to the final map resulting from the flow in (33), we choose to perform the rearrangement gradually along the flow. This results in a much smoother procedure: it takes small changes to alter the flow to make each point move toward its correct target, while fixing the map after the fact involves displacements of order one.

The strategy proposed here, similar to the one developed for constrained density estimation in [19], alternates between two kinds of steps: one that follows the direction of descent of $\hat{D}$ as before, and one that finds a direction along which the original cost decreases while the value of $\hat{D}$ does not deteriorate. Thus, for this second kind of steps, we consider the two objective functions

$$\hat{D}^k(\beta) = \frac{1}{m} \sum_{i=1}^m u_{\beta}^k(z_i^k) + \frac{1}{n} \sum_{j=1}^n (u_{\beta}^k)^*(y_j) \quad (61)$$

and

$$C^k(\beta) = \frac{1}{m} \sum_i^m |\nabla u_{\beta}^k(z_i^k) - x_i|^2, \quad (62)$$

Note: This equation follows directly from the earlier statement that the transport map is given by the gradient of the convex potential, $y = \nabla u$.

Understanding the Rearrangement Strategy and Equation (62)

The primary goal is to recover the optimal transport map—specifically, one that is the gradient of a convex function—in a manner that is numerically stable and computationally efficient. Rather than relying on direct constrained minimization (equation 60), which may be costly and lead to large displacements, the authors propose a gradual rearrangement method. This method alternates between two complementary types of updates:

- A gradient-based update that decreases the density-matching objective $\hat{D}^k(\beta)$.
- A cost-reducing update that aligns the map outputs $\nabla u_\beta^k(z_i^k)$ with the original positions x_i without increasing $\hat{D}^k(\beta)$.

To implement the second type of update, the paper introduces two objective functions:

Density-Matching Objective

$$\hat{D}^k(\beta) = \frac{1}{m} \sum_{i=1}^m u_\beta^k(z_i^k) + \frac{1}{n} \sum_{j=1}^n (u_\beta^k)^*(y_j)$$

This function measures how well the parameterized convex potential u_β^k transports mass from the source distribution ρ to the target distribution μ . The first term evaluates the potential at sample points from the source; the second term evaluates the convex conjugate (Legendre–Fenchel dual) at sample points from the target. Minimizing this objective encourages the learned map to align the pushforward of ρ with μ .

Proximity-to-Original Cost

$$C^k(\beta) = \frac{1}{m} \sum_{i=1}^m \|\nabla u_\beta^k(z_i^k) - x_i\|^2$$

This function penalizes deviations of the transport map from the original source locations. The gradient $\nabla u_\beta^k(z_i^k)$ gives the flow destination of particle z_i^k under the map. By minimizing $C^k(\beta)$, the method encourages a smoother, more stable map that avoids abrupt jumps, helping preserve structural coherence.

Relation to Permutation-Based Rearrangement

Earlier in the paper, the authors introduce a permutation-based strategy to improve the transport map. Given suboptimal outputs $\{w(x_i)\}$, one seeks a permutation π to minimize the cost:

$$\min_{\pi} \frac{1}{m} \sum_{i=1}^m \|w(x_{\pi(i)}) - x_i\|^2$$

subject to maintaining the empirical distribution of $\{w(x_i)\}$. This procedure enforces a structure that approximates a gradient of a convex function (as guaranteed by Brenier’s theorem).

Equation (62) can be interpreted as a continuous relaxation of this discrete permutation approach. Rather than finding a hard permutation π , the optimization over β continuously adjusts the transport map so that the output $\nabla u_\beta^k(z_i^k)$ moves closer to x_i .

Note: The update formula for z is precisely this expression, which makes sense since the transport map is defined as the gradient of the potential function u :

$$y(x) = \nabla u_\beta^k(x)$$

the Monte Carlo estimates of the functional $\tilde{D}$ and the L^2 cost respectively, with

$$u_{\beta}^k(z) = \frac{z^2}{2} + \sum_l \beta_l F_l^k(z)$$

as above, but with vectors β of dimension at least two (one is the minimal number of parameters for the first kind of steps.)

The cost-reducing step is described by the following algorithm, illustrated in figure 3:

1. Compute the gradients $\beta_D = -\nabla_{\beta} \hat{D}^k(\beta)|_{\beta=0}$ and $\beta_C = -\nabla_{\beta} C^k(\beta)|_{\beta=0}$
2. Find a direction $\bar{\beta}$ such that both $\beta_D \cdot \bar{\beta}$ and $\beta_C \cdot \bar{\beta}$ are positive. One option is to pick $\bar{\beta} = \beta_C$ when $\beta_D \cdot \beta_C > 0$, and otherwise the $\bar{\beta}$ in the line spanned by β_D and β_C such that $\beta_D \cdot \bar{\beta} = \beta_C \cdot \bar{\beta}$ as illustrated in figure (3). An alternative is described below when discussing convergence in the continuous setting.
3. Perform a descent move along the direction $\bar{\beta}$: $z^{k+1} = z^k + \epsilon \nabla_z \sum_l \bar{\beta}_l F_l^k(z)$. Here $\epsilon > 0$ can be a pre-established learning rate, or follow from Newton's method applied to the cost C , capped so that $\tilde{D}$ does not increase.

Then, as the density $\tilde{\rho}(z)$ underlying the evolving Lagrangian markers moves toward $\mu(z)$, the $z(x, t)$ are continuously re-arranged so the the cost is minimal.

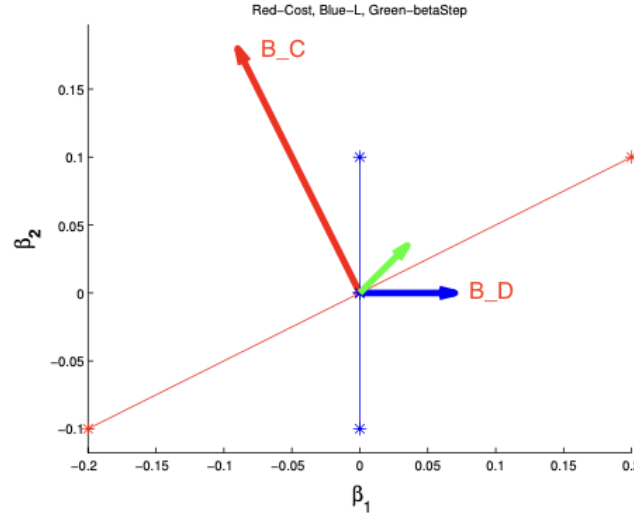


Figure 3: Direction of descent for the cost-decreasing step. The red and blue arrows represent β_C and β_D respectively; the thinner segments delimited by *'s are normal to these. The green arrow in the chosen direction of descent $\bar{\beta}$ has positive projection onto both β_C and β_D . This plot depicts the minimal situation with two free parameters β_l , but the general situation is entirely similar when looked at in the plane in β -space spanned by β_C and β_D .

The green arrow $\bar{\beta}$ in the figure represents a chosen descent direction in coefficient space. Each component $\bar{\beta}_l$ of this vector corresponds to a weight applied to the gradient of the associated basis function $F_l^k(z)$.

This descent direction is used directly in the particle update rule:

$$z^{k+1} = z^k + \epsilon \sum_l \bar{\beta}_l \nabla F_l^k(z),$$

where $\epsilon > 0$ is a small step size. This formula shifts each particle z^k in the direction of a learned transport map defined by the coefficients $\bar{\beta}$.

Thus, the green vector $\bar{\beta}$ selected in coefficient space becomes the actual set of weights determining how the particles move through space. This forms a complete feedback loop:

(geometric descent direction) $\Rightarrow$ parameter vector $\bar{\beta} \Rightarrow$ transport map $\Rightarrow$ updated particles.

Here, the notation $\bar{\beta}_l$ simply denotes the l -th component of the vector $\bar{\beta} \in \mathbb{R}^d$, where d is the number of basis functions F_l^k .